

GUIA PRÁTICO E DESAFIO

Construção de páginas HTML e CSS

com Box Model e Flexbox

Material do aluno + roteiro didático do professor

Produto final

Duas páginas conectadas por navegação: uma demonstração visual do Box Model e uma página de perfil organizada com Flexbox. O aluno construirá a solução progressivamente, compreenderá cada decisão e validará o resultado em diferentes larguras de tela.

Baseado nos arquivos: index.html, perfil.html e style.css fornecidos pelo professor.

Como usar este guia

Este material foi planejado como uma trilha prática. O aluno não começa copiando a solução completa. Ele monta a estrutura, aplica estilos em camadas, observa o efeito de cada propriedade e somente depois compara o resultado com a referência.

Público	Pré-requisitos	Entrega
Turma iniciante de HTML e CSS	Editor de código, navegador e noções de arquivos/pastas	index.html, perfil.html e style.css funcionando de forma responsiva

Objetivos de aprendizagem

- Reconhecer a função das principais tags semânticas do HTML5.
- Distinguir seletor de elemento, classe, ID, descendente e pseudo-classe.
- Explicar o Box Model: content, padding, border e margin.
- Compreender display: block, inline, inline-block e flex.
- Identificar flex container e flex items.
- Aplicar flex-direction, justify-content, align-items, gap, flex-wrap, flex e flex-shrink.
- Reutilizar valores com variáveis CSS.
- Construir layout responsivo com media query.
- Validar estrutura, aparência, navegação e acessibilidade básica.

Organização sugerida

Bloco	Foco	Resultado observável
1	Preparação e HTML semântico	Arquivos ligados e conteúdo estrutural visível
2	Fundamentos de CSS e Box Model	Página index estilizada e caixa didática pronta
3	Display e Flexbox	Cabeçalho, ações, painéis e perfil organizados

Exercício de fixação: Box Model, Display e Flexbox

4	Responsividade e validação	Layout adaptável e revisão final
5	Desafio autoral	Personalização sem quebrar os requisitos técnicos

Regra didática

Antes de escrever uma propriedade, o aluno deve responder: qual elemento será selecionado, qual comportamento será alterado e qual evidência visual confirmará que funcionou?

Mapa mental dos conceitos

HTML: estrutura e significado

HTML descreve a estrutura e o significado do conteúdo. CSS controla apresentação, espaçamento, alinhamento, cores e adaptação visual. Separar essas responsabilidades torna o projeto mais organizado e fácil de manter.

Tag	Papel no projeto
header	Cabeçalho e identidade da página
nav	Agrupar links de navegação
main	Conteúdo principal único
section	Seção temática
article	Conteúdo independente dentro de uma seção
h1, h2, h3	Hierarquia de títulos
p	Parágrafo
a	Navegação para outra página ou endereço
button	Ação executável
img	Imagem com texto alternativo
footer	Rodapé da página

CSS: caixa, fluxo e layout

Fluxo normal

Sem técnicas de layout, elementos de bloco ocupam a largura disponível e aparecem um abaixo do outro. Elementos inline permanecem na linha e normalmente não aceitam largura e altura da mesma forma que blocos.

display: inline-block combina participação na linha com a possibilidade de receber padding, border e dimensões. display: flex transforma o elemento em flex container; seus filhos diretos passam a ser flex items.

Flex container x flex items

Aplicado no container	Efeito principal
display: flex	Ativa o contexto flexível para os filhos diretos
flex-direction	Define o eixo principal: row ou column
justify-content	Distribui itens no eixo principal
align-items	Alinha itens no eixo transversal
gap	Cria espaçamento consistente entre itens
flex-wrap	Permite quebra para outra linha/coluna

Aplicado no item	Efeito principal
flex: 1	Permite crescer e dividir o espaço disponível
flex-shrink: 0	Impede que o item encolha
min-width	Define limite mínimo antes da quebra/adaptação

Desafio: reproduzir as duas telas

Sua missão é construir duas páginas conectadas, utilizando um único arquivo CSS. O objetivo não é apenas chegar a uma aparência semelhante, mas justificar tecnicamente cada escolha.

Critérios obrigatórios

- Cabeçalho com navegação entre as duas páginas.
- Página inicial com cartão explicativo e representação visual do Box Model.
- Painel com os conceitos Margin, Border e Padding.
- Página de perfil com foto, apresentação, botão e competências.
- Uso de HTML semântico e atributos básicos de acessibilidade.
- Uso de Box Model, inline-block e Flexbox.
- Responsividade para telas de até 43.75rem.
- Cores centralizadas em variáveis CSS.

Estrutura de pastas

```
projeto-html-css/
├── index.html
├── perfil.html
└── style.css
```

Dependência importante

Os três arquivos devem permanecer na mesma pasta. O href="style.css" conecta cada página ao CSS; href="index.html" e href="perfil.html" conectam a navegação.

Estratégia de trabalho

1. Criar primeiro a estrutura HTML sem se preocupar com a aparência.
2. Aplicar os estilos globais e validar se o CSS foi carregado.
3. Estilizar componentes do maior para o menor: página, cabeçalho, cartões, conteúdos e detalhes.
4. Ativar Flexbox somente após compreender quais são o container e os filhos diretos.
5. Testar cada etapa no navegador antes de avançar.
6. No final, reduzir a largura da janela e validar a media query.

Parte I: construção guiada

Etapa 1: Preparar os arquivos

Objetivo da etapa

Criar a base do projeto e garantir que a navegação e o CSS apontem para os nomes corretos.

O que o aluno fará

Crie a pasta do projeto e, dentro dela, os arquivos index.html, perfil.html e style.css.

Fundamentação técnica

Caminhos relativos procuram arquivos a partir da localização do documento atual. Por isso, href="style.css" funciona quando o CSS está no mesmo nível dos HTML.

Como verificar

Abra index.html no navegador. Use o DevTools, guia Network ou Elements, para confirmar que style.css foi carregado.

Roteiro do professor

Explique a pasta como uma pequena casa: as páginas são cômodos e os links são portas. Se a placa da porta tiver outro nome, o navegador não encontrará o destino.

Etapa 2: Criar o esqueleto HTML5

Objetivo da etapa

Definir documento, idioma, codificação, viewport, título e vínculo com o CSS.

O que o aluno fará

Repita a base em cada arquivo HTML, alterando apenas o title e o conteúdo do body.

Fundamentação técnica

DOCTYPE informa o modo HTML5. lang="pt-BR" auxilia leitores de tela. charset="UTF-8" suporta acentos. viewport permite que o layout acompanhe a largura do dispositivo.

Como verificar

O título deve aparecer na aba do navegador, e os caracteres acentuados devem ser exibidos corretamente.

Roteiro do professor

Pergunte: qual parte aparece na tela e qual parte configura o documento? Diferencie head de body antes de inserir componentes.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Título da página</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Conteúdo visível -->
</body>
</html>
```

Etapa 3: Montar o cabeçalho e a navegação

Objetivo da etapa

Usar header e nav para criar identidade e acesso entre as páginas.

O que o aluno fará

No index, use h1 com id titulo-principal. No perfil, use p com classe logo. Em ambos, crie nav.menu com dois links e marque o link da página atual com class="ativo".

Fundamentação técnica

header representa conteúdo introdutório. nav identifica um bloco de navegação. O ID é único; classes podem ser reutilizadas. aria-label dá um nome compreensível à navegação.

Como verificar

Clique nos dois links e confirme que a troca de páginas acontece sem erro 404.

Roteiro do professor

Mostre que ativo não é uma tag nem um comportamento automático. É uma classe escolhida para receber estilo visual no CSS.

```
<header class="cabecalho">
  <h1 id="titulo-principal">Explorando o CSS3</h1>
  <nav class="menu" aria-label="Navegação principal">
    <a href="index.html" class="ativo">Box Model</a>
    <a href="perfil.html">Perfil com Flexbox</a>
  </nav>
</header>
```

Etapa 4: Estruturar a página do Box Model

Objetivo da etapa

Organizar conteúdo principal em seções semânticas.

O que o aluno fará

Dentro de main.conteudo-principal, crie section.cartao e section.painel-conceitos. No cartão, adicione etiqueta, título, descrição, demonstração e ações. No painel, crie três articles.

Fundamentação técnica

section agrupa um tema; article representa um bloco que mantém sentido de forma independente. A hierarquia h1, h2 e h3 facilita leitura e navegação assistiva.

Como verificar

Desative temporariamente o CSS. O conteúdo deve continuar em ordem lógica e compreensível.

Roteiro do professor

Diga aos alunos: um bom HTML se sustenta mesmo sem maquiagem. O CSS muda a aparência, mas a informação deve continuar organizada.

Etapa 5: Representar o Box Model com caixas aninhadas

Objetivo da etapa

Materializar visualmente as quatro camadas de uma caixa CSS.

O que o aluno fará

Crie `div.box-model` e, dentro dela, as camadas `margin`, `border`, `padding` e `content`, cada uma envolvendo a próxima.

Fundamentação técnica

Todo elemento é desenhado como uma caixa. `Content` é a área do conteúdo; `padding` é o espaço interno; `border` contorna conteúdo e `padding`; `margin` separa a caixa dos vizinhos. O aninhamento reproduz visualmente essa ordem.

Como verificar

No DevTools, selecione uma camada e observe o diagrama de Box Model na guia `Computed`.

Roteiro do professor

Use a analogia de um quadro: a imagem é `content`, o espaço entre imagem e moldura é `padding`, a moldura é `border` e o espaço até outro quadro é `margin`.

```
<div class="box-model" aria-label="Demonstração do Box Model">
  <div class="camada-margin">Margin
    <div class="camada-border">Border
      <div class="camada-padding">Padding
        <div class="camada-content">Content</div>
      </div>
    </div>
  </div>
</div>
```

Etapa 6: Criar ações com link e botão

Objetivo da etapa

Comparar navegação com ação e preparar o uso de `inline-block` e `flex`.

O que o aluno fará

Adicione um link com classe `botao` e um button com as classes `botao` e `botao-secundario` dentro de `div.acoes`.

Fundamentação técnica

O link leva a outro recurso. O button representa uma ação. Uma classe base concentra o estilo comum, enquanto a classe modificadora altera apenas o necessário.

Como verificar

Use `Tab` para navegar. Verifique se link e botão recebem foco e se o link abre `perfil.html`.

Roteiro do professor

Questione: se os dois parecem botões, por que não usar a mesma tag? A resposta está na semântica e no comportamento esperado.

Etapa 7: Estruturar a página de perfil

Objetivo da etapa

Criar foto, apresentação e lista de competências com marcação semântica.

O que o aluno fará

Em main.pagina-perfil, crie section.card-perfil. Dentro dela, adicione div.resumo-perfil com img e div.apresentacao. Depois, divisor e seção de competências.

Fundamentação técnica

alt descreve a finalidade da imagem. target="_blank" abre nova guia; rel="noopener noreferrer" reduz riscos de acesso da nova página ao contexto anterior. aria-labelledby associa a seção ao seu título.

Como verificar

A imagem deve carregar, o texto alternativo deve existir e o link externo deve abrir em outra guia.

Roteiro do professor

Peça aos alunos para lerem apenas as tags e preverem a estrutura visual antes de aplicar Flexbox.

Etapa 8: Aplicar reset e box-sizing

Objetivo da etapa

Remover diferenças básicas de espaçamento e tornar as medidas previsíveis.

O que o aluno fará

No início do CSS, aplique seletor universal com box-sizing, margin e padding.

Fundamentação técnica

Por padrão, content-box soma padding e border à largura declarada. border-box faz a largura incluir conteúdo, padding e border, facilitando o cálculo do layout. O reset remove margens padrão do navegador.

Como verificar

Inspecione um cartão e confirme no DevTools que box-sizing calculado é border-box.

Roteiro do professor

Mostre uma caixa de 300px com padding. Pergunte se ela continuará com 300px. Compare content-box e border-box ao vivo.

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

Etapa 9: Criar variáveis CSS

Objetivo da etapa

Centralizar as cores para facilitar manutenção e consistência.

O que o aluno fará

Defina as custom properties em :root e use var(--nome) nas regras seguintes.

Fundamentação técnica

:root representa o elemento raiz do documento. Variáveis CSS reduzem repetição; uma alteração em um valor pode atualizar vários componentes.

Como verificar

Troque temporariamente --cor-secundaria e observe todos os componentes que utilizam a variável.

Roteiro do professor

Compare variáveis com etiquetas em potes de tinta: componentes pedem a tinta pelo nome, sem memorizar o código hexadecimal.

```
:root {
  --cor-primaria: #0d3b66;
  --cor-secundaria: #2a9d8f;
  --cor-fundo: #eef2f5;
  --cor-texto: #333333;
  --cor-suave: #667085;
  --branco: #ffffff;
}
```

Etapa 10: Estilizar body e fluxo vertical**Objetivo da etapa**

Definir aparência global e manter o rodapé após o conteúdo.

O que o aluno fará

Configure fundo, texto, família tipográfica, line-height e min-height. Depois transforme body em flex container com direção column.

Fundamentação técnica

min-height: 100vh faz a página ocupar no mínimo a altura da janela. flex-direction: column torna o eixo principal vertical. O main poderá receber flex: 1 e ocupar o espaço restante.

Como verificar

Em uma tela alta com pouco conteúdo, o rodapé deve permanecer na parte inferior sem posicionamento absoluto.

Roteiro do professor

Desenhe o body como uma coluna com três itens: header, main e footer. Mostre que flex: 1 no main absorve o espaço livre.

```
body {
  background-color: var(--cor-fundo);
  color: var(--cor-texto);
  font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
  line-height: 1.6;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}
```

Etapa 11: Transformar o cabeçalho em flex container**Objetivo da etapa**

Posicionar identidade e menu nas extremidades e centralizar verticalmente.

O que o aluno fará

Aplique display:flex, justify-content:space-between, align-items:center e gap na classe cabecalho.

Fundamentação técnica

Com flex-direction padrão row, o eixo principal é horizontal. justify-content atua horizontalmente; align-items atua verticalmente. gap cria distância mínima sem margens individuais.

Como verificar

A identidade deve ficar à esquerda e o menu à direita, alinhados pelo centro.

Roteiro do professor

Peça que os alunos apontem o eixo principal e o transversal. Depois troque flex-direction para column e repita a pergunta.

```
.cabecalho {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}
```

Etapa 12: Estilizar menu e estados**Objetivo da etapa**

Praticar seletor descendente, inline-block e pseudo-classe.

O que o aluno fará

Selecione .menu a, remova sublinhado, aplique cor, padding e border-radius. Em .menu a:hover e .menu a.ativo, mude o fundo.

Fundamentação técnica

.menu a alcança apenas links descendentes de .menu. inline-block permite padding e área visual de botão sem transformar cada link em uma linha inteira. :hover reage ao ponteiro; .ativo representa estado persistente.

Como verificar

Passe o mouse e confirme o feedback visual. Depois alterne a classe ativo entre os links.

Roteiro do professor

Diferencie interação temporária de estado da aplicação. Hover existe enquanto o ponteiro está sobre o item; ativo permanece porque foi escrito no HTML.

Etapa 13: Dimensionar conteúdo com unidades relativas**Objetivo da etapa**

Criar largura fluida com limite máximo e espaçamento centralizado.

O que o aluno fará

Aplique width:90%, min-height:70vh, margin:2rem auto e flex:1 aos mains. Defina max-width específico para cada página.

Fundamentação técnica

% responde ao espaço do elemento pai; rem acompanha o tamanho raiz da fonte; vh é relativo à altura da janela. margin auto nas laterais centraliza um bloco com largura definida. max-width evita linhas excessivamente longas.

Como verificar

Aumente e reduza a janela: a página deve encolher até 90%, mas não ultrapassar o max-width.

Roteiro do professor

Use a metáfora de uma faixa elástica com trava: width em porcentagem estica; max-width impede crescimento infinito.

Etapa 14: Construir o cartão com Box Model

Objetivo da etapa

Combinar largura, border, radius, padding, margin e shadow.

O que o aluno fará

Estilize .cartao e .card-perfil em uma regra agrupada. Aplique fundo, borda, raio, preenchimento, centralização e sombra.

Fundamentação técnica

O seletor agrupado evita duplicação. A sombra não altera o tamanho geométrico da caixa, apenas a pintura. border-radius arredonda os cantos. Padding impede o conteúdo de encostar na borda.

Como verificar

No DevTools, desative padding, border e box-shadow individualmente e descreva o efeito de cada propriedade.

Roteiro do professor

Faça a leitura externa para interna: margin separa o cartão; border delimita; padding respira; content comunica.

```
.cartao,
.card-perfil {
  width: 100%;
  max-width: 56.25rem;
  background-color: var(--branco);
  border: 0.125rem solid var(--cor-secundaria);
  border-radius: 1rem;
  padding: 1.75rem;
  margin: 0 auto;
  box-shadow: 0 0.75rem 1.875rem rgba(13, 59, 102, 0.15);
}
```

Etapa 15: Colorir as camadas do Box Model

Objetivo da etapa

Associar cada camada a uma evidência visual.

O que o aluno fará

Aplique fundos diferentes, padding progressivo e tipos de borda distintos nas classes camada-margin, camada-border, camada-padding e camada-content.

Fundamentação técnica

As cores não são a definição do Box Model; são apenas uma representação pedagógica. O comportamento real é produzido por margin, border e padding.

Como verificar

Altere apenas o padding de uma camada e observe quais regiões crescem.

Roteiro do professor

Evite que os alunos confundam cor com conceito. Pergunte: se removermos o fundo, padding continua existindo? Sim.

Etapa 16: Criar botões e transições

Objetivo da etapa

Converter links e botões em componentes visuais reutilizáveis.

O que o aluno fará

Estilize `.botao` com `inline-block`, cores, padding, border, radius, `font:inherit` e cursor. Adicione `transition` e estado `hover`.

Fundamentação técnica

`font: inherit` reduz diferenças de fonte entre `button` e `link`. `transition` interpola mudanças de propriedades ao longo do tempo. `transform: translateY` desloca visualmente sem reorganizar o fluxo.

Como verificar

Passa o mouse e observe cor e deslocamento suaves. Use DevTools para desativar `transition` e comparar.

Roteiro do professor

Demonstre que `transition` não cria o estado; apenas suaviza a passagem entre estado normal e `hover`.

Etapa 17: Organizar ações e painéis com Flexbox

Objetivo da etapa

Distribuir elementos lado a lado com quebra controlada.

O que o aluno fará

Em `.acoes`, use `display:flex`, `flex-wrap:wrap` e `gap`. Em `.painel-conceitos`, use `display:flex` e `gap`. Nos `articles`, aplique `flex:1`.

Fundamentação técnica

`flex-wrap` permite que os itens migrem para outra linha quando não couberem. `flex:1` faz os artigos compartilharem o espaço disponível de forma equivalente.

Como verificar

Reduza a largura. Os botões podem quebrar de linha sem sobreposição; os artigos mantêm proporção em telas amplas.

Roteiro do professor

Substitua `gap` por margens para mostrar a complexidade adicional. Explique que `gap` pertence ao layout e mantém espaçamento uniforme.

Etapa 18: Construir o resumo de perfil com Flexbox

Objetivo da etapa

Alinhar foto e apresentação como itens de um mesmo container.

O que o aluno fará

Aplique `display:flex`, `align-items:center` e `gap` em `.resumo-perfil`. Na imagem, defina dimensões, `border-radius:50%`, `object-fit:cover`, borda e `flex-shrink:0`.

Fundamentação técnica

A imagem e `.apresentacao` são filhos diretos, portanto `flex items`. `object-fit: cover` preserva proporção enquanto preenche a área, recortando excedentes. `flex-shrink:0` impede deformação por falta de espaço.

Como verificar

A foto deve permanecer circular e com 7.5rem, mesmo quando o texto for longo.

Roteiro do professor

Compare uma moldura redonda: width e height iguais criam o quadrado-base; border-radius:50% cria o círculo; object-fit decide como a foto cabe na moldura.

```
.resumo-perfil {
  display: flex;
  align-items: center;
  gap: 1.5rem;
}
.resumo-perfil img {
  width: 7.5rem; height: 7.5rem;
  border-radius: 50%;
  object-fit: cover;
  flex-shrink: 0;
}
```

Etapa 19: Distribuir competências

Objetivo da etapa

Criar uma linha de cartões de tamanhos coerentes.

O que o aluno fará

Transforme .habilidades em flex container com direction row e gap. Em .habilidade, use flex:1, min-width, padding e novo flex container em coluna.

Fundamentação técnica

Há flex containers aninhados: .habilidades organiza os cartões; cada .habilidade organiza seu strong e span. justify-content:center atua verticalmente porque a direção do item é column.

Como verificar

Os três cartões devem ocupar larguras equivalentes. Adicione texto maior em um item e observe a adaptação.

Roteiro do professor

Destaque que um flex item também pode ser flex container. O papel depende da relação: filho do container externo e pai dos elementos internos.

Etapa 20: Aplicar responsividade

Objetivo da etapa

Adaptar o layout sem duplicar HTML.

O que o aluno fará

Crie @media (max-width:43.75rem). Mude cabeçalho, resumo, painel e habilidades para flex-direction:column. Centralize textos e permita wrap no menu.

Fundamentação técnica

Media query aplica regras somente quando a condição é verdadeira. A cascata mantém regras anteriores e sobrescreve apenas propriedades declaradas depois.

Como verificar

Abra o modo responsivo do navegador. Teste acima e abaixo de 700px, equivalente a 43.75rem quando a raiz é 16px.

Roteiro do professor

Não apresente responsividade como outra página. É o mesmo conteúdo recebendo outra estratégia de layout conforme o espaço disponível.

```
@media (max-width: 43.75rem) {
  .cabecalho, .resumo-perfil, .painel-conceitos, .habilidades {
    flex-direction: column;
  }
  .menu {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
  }
}
```

Etapa 21: Validar e depurar

Objetivo da etapa

Desenvolver rotina de investigação em vez de tentativa aleatória.

O que o aluno fará

Revise Console, Elements, Styles e Computed; verifique caminhos, seletores, propriedades riscadas e dimensões.

Fundamentação técnica

A cascata decide qual declaração vence por origem, importância, especificidade e ordem. Uma regra riscada no DevTools não significa CSS inválido; pode ter sido sobrescrita.

Como verificar

Não pode haver erro de arquivo não encontrado. Links devem funcionar e nenhum conteúdo pode sobrepor outro em tela estreita.

Roteiro do professor

Modele o raciocínio em voz alta: 1) o seletor encontrou o elemento? 2) a propriedade é válida? 3) outra regra venceu? 4) o elemento é realmente filho direto do flex container?

Parte II: roteiro didático do professor

O roteiro abaixo organiza a mediação em momentos curtos de observação, hipótese, implementação, teste e explicação. A prioridade é fazer o aluno verbalizar o motivo das escolhas.

Abertura da aula

Fala sugerida

Hoje não vamos apenas copiar uma tela. Vamos desmontar o layout em decisões: qual tag dá significado, qual caixa cria o espaçamento, qual elemento controla o alinhamento e como o navegador reage quando a largura muda. Ao final, vocês deverão conseguir explicar a solução.

Perguntas diagnósticas

- O que acontece quando abrimos um HTML sem CSS?
- Qual diferença vocês esperam entre margin e padding?
- Quem controla o alinhamento com Flexbox: o pai, o filho ou ambos?
- Por que usar header, nav, main e section em vez de somente div?
- O que deve acontecer com três cartões lado a lado quando a tela ficar estreita?

Ciclo de demonstração para cada etapa

Momento	Ação do professor	Ação do aluno
---------	-------------------	---------------

1. Prever	Mostra o problema e pergunta o resultado esperado.	Formula hipótese antes do código.
2. Construir	Escreve poucas linhas e explica seletor e propriedade.	Reproduz e comenta o código.
3. Observar	Atualiza navegador e usa DevTools.	Identifica a evidência visual.
4. Variar	Altera um valor por vez.	Compara antes e depois.
5. Explicar	Solicita justificativa técnica.	Explica com vocabulário correto.

Demonstrações essenciais

Box Model ao vivo

Selecione o cartão no DevTools. Ligue e desligue padding, border e margin. Peça que a turma descreva qual região mudou.

Eixos do Flexbox

No cabeçalho, alterne row e column. Mostre que o significado visual de justify-content e align-items depende dos eixos.

Filho direto

Envolve temporariamente um item em outra div. Mostre que apenas filhos diretos se tornam flex items daquele container.

Cascata

Crie duas regras para a mesma propriedade. Inverta a ordem e depois aumente a especificidade.

Responsividade

Reduza a viewport lentamente e peça que os alunos identifiquem o ponto em que a media query passa a valer.

Plano de condução em blocos

Bloco	Condução	Evidência de aprendizagem
Preparação	Monte a pasta, explique caminhos relativos e valide o vínculo CSS.	Aluno identifica causa de CSS não carregado.
HTML semântico	Construa a hierarquia antes da estilização. Leia a página sem CSS.	Aluno justifica ao menos cinco tags.
Box Model	Faça o cartão e a demonstração com DevTools.	Aluno diferencia espaço interno e externo.
Display	Compare block, inline e inline-block nos links.	Aluno prevê quebra de linha e área de caixa.

Flexbox	Aplique ao cabeçalho, ações, painéis e perfil.	Aluno aponta container, itens e eixos.
Responsividade	Ative a media query e teste larguras.	Aluno explica sobrescrita pela cascata.
Desafio	Remova parte do apoio e peça personalização controlada.	Aluno reproduz e apresenta decisões.

Erros frequentes e intervenções

Sintoma	Causa provável	Intervenção didática
CSS não aparece	Nome/caminho incorreto ou link ausente	Conferir pasta e href no Elements/Network.
Flexbox não alinha o item esperado	display:flex aplicado ao elemento errado	Marcar no HTML quem é pai e quem são filhos diretos.
justify-content parece não funcionar	Não há espaço livre ou eixo foi confundido	Adicionar altura/largura temporária e desenhar os eixos.
Foto fica oval ou espremida	Largura e altura diferentes ou shrink ativo	Igualar dimensões e usar flex-shrink:0.
Cartão ultrapassa a tela	Box Model ou largura rígida	Verificar border-box, width, max-width e padding.
Regra aparece riscada	Outra declaração venceu na cascata	Comparar especificidade e ordem no DevTools.
Media query não entra	Viewport ausente ou condição não atendida	Conferir meta viewport e largura calculada.

Parte III: exercício de fixação

Rodada A: complete sem consultar a solução

- Crie o header e a nav das duas páginas.
- Construa o cartão principal da index com etiqueta, título e descrição.
- Represente as quatro camadas do Box Model com divs aninhadas.
- Crie três artigos para Margin, Border e Padding.
- Monte o resumo do perfil com imagem e texto.
- Crie três cartões de competências.
- Ligue as páginas por links relativos.

Rodada B: aplique os conceitos obrigatórios

Requisito	Onde aplicar	Pergunta de justificativa
box-sizing:border-box	Reset universal	O que passa a fazer parte da

		largura?
inline-block	Links/botões	Por que padding funciona de forma previsível?
display:flex	Cabeçalho, ações, perfil e habilidades	Quem são os filhos diretos?
justify-content	Cabeçalho e habilidades	Qual é o eixo principal aqui?
align-items	Cabeçalho e resumo	Qual é o eixo transversal?
flex-wrap	Ações e menu móvel	O que acontece quando falta espaço?
flex:1	Painéis e habilidades	Como o espaço é dividido?
media query	Telas estreitas	Quais regras são sobrescritas?

Rodada C: personalize sem quebrar

- Troque a paleta alterando apenas variáveis CSS.
- Substitua nome, biografia, foto e competências.
- Adicione uma quarta competência e preserve a responsividade.
- Crie um terceiro link de navegação fictício e faça o menu quebrar corretamente.
- Mude o breakpoint e registre o motivo da escolha.

Restrição do desafio

Não vale duplicar todo o CSS para cada página nem usar posicionamento absoluto para montar o layout principal. A solução deve continuar baseada no fluxo normal, Box Model e Flexbox.

Perguntas de compreensão

1. Explique por que border-box facilita o dimensionamento.

Resposta: _____

2. Qual a diferença entre margin e padding?

Resposta: _____

3. O que transforma um elemento em flex container?

Resposta: _____

4. Quais elementos se tornam flex items?

Resposta: _____

5. Como flex-direction altera os eixos?

Resposta: _____

6. Quando justify-content e align-items parecem trocar de papel?

Resposta: _____

7. Qual a diferença entre gap e margin entre itens?

Resposta: _____

8. Por que flex-wrap é importante?

Resposta: _____

9. O que flex:1 faz nos artigos?

Resposta: _____

10. Por que usar flex-shrink:0 na imagem?

Resposta: _____

11. O que object-fit:cover resolve?

Resposta: _____

12. Qual diferença entre .menu a e #titulo-principal?

Resposta: _____

13. O que :hover representa?

Resposta: _____

14. Por que a classe ativo é necessária?

Resposta: _____

15. Como a media query utiliza a cascata?

Resposta: _____

16. Por que nav deve ter um nome acessível?

Resposta: _____

17. Quando usar a em vez de button?

Resposta: _____

18. Por que max-width melhora a leitura?

Resposta: _____

19. Qual vantagem das variáveis CSS?

Resposta: _____

20. Como investigar uma propriedade riscada no DevTools?

Resposta: _____

Gabarito comentado

1. A largura declarada passa a incluir conteúdo, padding e border, reduzindo somas inesperadas.

2. Padding é espaço interno entre conteúdo e borda; margin é espaço externo entre a caixa e seus vizinhos.
3. A declaração `display:flex` ou `display:inline-flex`.
4. Os filhos diretos do flex container.
5. Define o eixo principal como linha ou coluna; o eixo transversal fica perpendicular.
6. Quando `flex-direction` muda, porque `justify-content` sempre atua no eixo principal e `align-items` no transversal.
7. `gap` pertence ao container e cria espaço uniforme entre itens; `margin` pertence a cada item e pode afetar bordas externas.
8. Permite acomodar itens em novas linhas/colunas quando o espaço não é suficiente.
9. Faz os artigos crescerem e compartilhar o espaço disponível.
10. Impede que a imagem encolha e perca a dimensão circular planejada.
11. Preenche a moldura preservando a proporção, com recorte do excedente.
12. `.menu a` é um seletor descendente reutilizável; `#titulo-principal` seleciona um ID único.
13. Um estado de interação enquanto o ponteiro está sobre o elemento.
14. Para indicar visualmente, de forma persistente, a página atual.
15. As regras dentro da condição entram depois e sobrescrevem propriedades específicas, mantendo o restante.
16. Para que tecnologias assistivas identifiquem a finalidade daquele conjunto de links.
17. `a` para navegação; `button` para executar uma ação.
18. Evita linhas excessivamente longas e crescimento desnecessário em telas grandes.
19. Reúso, consistência e manutenção centralizada.
20. Verificar quem sobrescreveu, comparar ordem e especificidade e confirmar se a propriedade é válida.

Checklist de validação

HTML

- ☐ `DOCTYPE`, `lang`, `charset` e `viewport` presentes.
- ☐ Um `h1` principal por página.
- ☐ `header`, `nav`, `main`, `section/article` e `footer` utilizados com sentido.
- ☐ Links relativos funcionam.
- ☐ Imagem possui `alt`.
- ☐ Navegação possui `aria-label`.
- ☐ Seção de competências está associada ao título.

CSS

- ☐ Reset universal e `border-box` aplicados.
- ☐ Variáveis CSS usadas para a paleta.
- ☐ Cartões apresentam `border`, `padding`, `margin` e `shadow`.
- ☐ O Box Model visual contém quatro camadas.
- ☐ Cabeçalho, ações, painéis, resumo e habilidades usam Flexbox.
- ☐ `Gap`, `wrap`, `flex` e `flex-shrink` aparecem em contexto adequado.
- ☐ `Hover` e `transition` fornecem feedback.
- ☐ Media query reorganiza o conteúdo.

Teste visual e funcional

- [] Nenhum texto está cortado.
- [] Nenhum componente se sobrepõe.
- [] O rodapé permanece após o conteúdo.
- [] A imagem permanece circular.
- [] A navegação indica a página atual.
- [] Layout funciona em largura ampla e estreita.
- [] Console sem erros relevantes.

Rubrica de avaliação

Critério	Excelente	Adequado	Em desenvolvimento
Estrutura HTML	Semântica consistente, hierarquia clara e atributos de apoio.	Estrutura correta com pequenas inconsistências.	Uso excessivo de div ou hierarquia confusa.
Box Model	Explica e aplica corretamente todas as camadas.	Aplica, mas justifica parcialmente.	Confunde espaços internos e externos.
Flexbox	Identifica containers, itens, eixos e propriedades em todos os blocos.	Layout funciona com pequenos ajustes.	Aplica flex ao elemento errado ou depende de tentativa.
CSS e cascata	Seletores organizados, variáveis e reuso de classes.	CSS funcional com alguma repetição.	Regras duplicadas ou conflitos não compreendidos.
Responsividade	Adaptação limpa, sem corte ou sobreposição.	Adaptação funcional com detalhes visuais.	Layout quebra em tela estreita.
Explicação técnica	Justifica decisões com vocabulário correto.	Explica o essencial.	Apresenta apenas o resultado sem fundamentação.

Entrega do aluno

- Pasta com os três arquivos.
- Captura das duas páginas em tela ampla.
- Captura das duas páginas em tela estreita.
- Breve comentário no CSS identificando pelo menos oito conceitos.
- Apresentação oral ou escrita explicando um flex container e um Box Model.

Referência técnica dos seletores do projeto

Seletor	Tipo	Responsabilidade
*	Universal	Reset e box-sizing
:root	Pseudo-classe raiz	Variáveis CSS
body	Elemento	Aparência e estrutura vertical
.cabecalho	Classe	Flex container do topo
#titulo-principal	ID	Título único da index
.menu a	Descendente	Links dentro do menu
.menu a:hover	Pseudo-classe	Interação do ponteiro
.cartao, .card-perfil	Agrupado	Base visual compartilhada
.acoes	Classe	Container flexível dos botões
.painel-conceitos article	Descendente	Itens conceituais
.resumo-perfil	Classe	Container da foto e apresentação
.habilidades	Classe	Container dos cartões técnicos
@media (...)	Regra condicional	Adaptação por largura

Vocabulário mínimo

Seletor

Expressão que localiza elementos HTML.

Declaração

Par propriedade: valor.

Regra CSS

Seletor acompanhado do bloco de declarações.

Cascata

Processo que determina qual declaração vence.

Especificidade

Peso de um seletor na disputa da cascata.

Herança

Transmissão de determinadas propriedades do pai para descendentes.

Viewport

Área visível da página no dispositivo.

Breakpoint

Ponto em que uma condição responsiva muda o layout.

Eixo principal

Eixo definido por flex-direction.

Eixo transversal

Eixo perpendicular ao principal.

Encerramento e reflexão

Fala sugerida

A tela pronta é apenas a evidência final. O aprendizado está em conseguir olhar para um layout, dividi-lo em caixas e relações, escolher tags com significado e decidir conscientemente qual elemento deve controlar o alinhamento.

Reflexão do aluno

Qual conceito ficou mais claro quando você alterou valores no DevTools?

Em qual momento você confundiu o container com o item?

Que propriedade resolveu um problema real de adaptação?

O que você faria diferente se a página tivesse seis competências?

Como explicaria Box Model e Flexbox para alguém que nunca programou?

Desafio extra

Crie uma terceira página chamada projetos.html com três cartões de projeto. Reutilize cabeçalho, menu, botão, cartão e paleta. O layout deve ter três colunas em tela ampla e uma coluna em tela estreita. Documente no CSS quais regras foram reutilizadas e quais precisaram ser novas.

Critério de sucesso

O aluno concluiu o desafio quando consegue reproduzir as telas, explicar a função das tags e propriedades principais, apontar flex containers e flex items e corrigir um erro usando o DevTools.